Smart Home Energy Management System (SHEMS)
Milestone 1: Authentication & Role Based Access Control (RBAC)

1. Module-1 Overview

Module–1 is the security entry point of SHEMS.
In our project we implemented:

1. Registration (new user creation)
2. Login (authentication)
3. Role Based Access Control (authorization using roles)
4. Role Based Redirection (USER → dashboard, ADMIN → admin screen)
5. Forgot Password and Reset Password

This module ensures that only authenticated users enter the system and access is restricted based on their role.

2)

2.1 Authentication

Authentication = verifying identity of the user using credentials (email + password).
If credentials are correct → user is logged in.
Implementation in our project:

1. User enters email + password in UI page:
   - src/main/resources/templates/login.html
2. Request goes through Spring Security filter chain and verifies credentials using:
   - SpringSecurityConfig.java
3. User details are validated using database user record:
   - UserRepository.java (fetch user from DB)
   - Password is validated using BCrypt matching
4. On successful login:
   - Spring creates an authenticated session.

Result: User becomes logged-in.

2.2 Authorization

Authorization = checking permission of authenticated user.
A logged-in user should not automatically access everything.
Example:

- USER role → allowed only USER dashboard
- ADMIN role → allowed ADMIN dashboard
- **Implementation in our project:**

Authorization rules are configured inside:

- SpringSecurityConfig.java

Access separation is implemented using role mappings in DB tables:

- role
- users_role

Result: Role decides what user can access.

2.3 RBAC (Role Based Access Control)

RBAC means the system restricts actions based on role.

Implementation in our project:

Role mapping stored in DB:

users_role table links:

- cust_id (user id)
- role_id (role id)

This role is used by Spring Security internally when user logs in.

Result: Role is assigned and checked automatically.

2.4 MVC Pattern

MVC is used to separate code cleanly.

- View → HTML pages
- Controller → handles routes
- Service → business logic
- Repository → DB operations
- Model → DB entities

Our implementation:

View (UI pages):

- templates/login.html
- templates/register.html
- templates/dashboard.html
- templates/adminScreen.html
- templates/forgot-password.html
- templates/reset-password.html
- templates/message.html

Controller (routing):

- LoginController.java
- RegistrationController.java
- DashboardController.java
- AdminController.java
- ForgotPasswordController.java

Service (logic):

- Service classes inside service/ package (registration, reset flow, encryption logic)

Repository (DB):

- UserRepository.java
- RoleRepository.java
- PasswordResetTokenRepository.java

Model (entities):

- classes inside model/
- User entity
- Role entity
- Token entity (reset password)

3. System Layer Explanation

3.1 Presentation Layer (UI Layer)

Collects input and shows output to user.

- **Our code:**
- All UI pages are inside:
    - src/main/resources/templates/

Example:

- User enters login credentials in login.html
- User enters registration details in register.html

3.2 Controller Layer

Controller acts as a bridge between UI and backend logic.

- **Our code:**
- All controllers inside:
    - com.role.implementation.controller

Examples:

- Registration request handled by RegistrationController.java
- Login page route handled by LoginController.java
- User dashboard routing by DashboardController.java
- Admin routing by AdminController.java
- Forgot/reset by ForgotPasswordController.java

3.3 Service Layer

Service layer contains real business logic.

- **Our code:**
- Inside com.role.implementation.service
- Responsibilities:
    - encrypt password
    - create user
    - assign role
    - generate reset token
    - update password

3.4 Repository Layer

Repository handles DB operations.

- **Our code:**
- Inside com.role.implementation.repository

Example:

- Fetch user by email:
    - UserRepository.java

Example:

- Fetch role:
    - RoleRepository.java

Example:

- Forgot password token saved/validated:

- PasswordResetTokenRepository.java

3.5 Database Layer

Stores persistent user credentials and role mapping.

Our DB tables:

- user
- role
- users_role
- hibernate_sequence
- reset token table (if enabled)

4. Module–1 Functional Flow

4.1 Registration Flow (Register user)

Register should:

- validate input
- encrypt password
- save user details
- assign role

Implementation flow in our code:

1. UI page:
   - register.html
2. Controller:
   - RegistrationController.java
3. Service:
   - Encrypt password using BCrypt
   - Save user to DB
   - Save role mapping to DB
4. DB updated:
   - user table insert
   - role table has roles
   - users_role table updated with mapping

Output: user registered successfully.

4.2 Login Flow (Authentication)

User logs in with email + password.

If correct → allow.

Implementation flow:

1. UI page:
   - login.html
2. Spring Security validates (configured in):
   - SpringSecurityConfig.java
3. DB check:
   - UserRepository.findByEmail(...)

Output: user is authenticated.

4.3 Role Based Redirect Flow (Authorization result)

After authentication:

- check role
- redirect user to correct page

Implementation:

Implemented in:

- CustomSuccessHandler.java

Logic:

- If role = USER → /dashboard
- If role = ADMIN → /adminScreen

UI Pages shown:

- dashboard.html
- adminScreen.html

Output: role-based redirection working.

4.4 Forgot Password Flow

Forgot password should:

- take email
- generate reset token
- allow user to reset password

Implementation:

1. UI:
   - forgot-password.html
2. Controller:
   - ForgotPasswordController.java
3. Repository:
   - PasswordResetTokenRepository.java
4. Reset page:
   - reset-password.html

Output: password reset supported.

5. Project Folder Structure

📁 src/main/java/com/role/implementation/

📁 config/

- SpringSecurityConfig.java
- CustomSuccessHandler.java

📁 controller/

- AdminController.java
- DashboardController.java
- ForgotPasswordController.java
- LoginController.java
- RegistrationController.java

📁 DTO/

- UserLoginDTO.java

- UserRegisteredDTO.java

📁 model/

- User / Role / Token entity classes

📁 repository/

- PasswordResetTokenRepository.java
- RoleRepository.java
- UserRepository.java

📁 service/

- business logic classes

📁 src/main/resources/templates/

- adminScreen.html
- dashboard.html
- forgot-password.html
- login.html
- message.html
- register.html
- reset-password.html

6. Database Verification

To prove implementation is correct, run these queries in MySQL:

```
USE role_auth;

SELECT id,email,name FROM `user`;
SELECT * FROM role;
SELECT * FROM users_role;
```

Expected:

- user table contains registered users
- role table contains USER and ADMIN
- users_role contains mapping between users and roles

HOW TO RUN AUTHENTICATION PROJECT

Step 0: Install Required Software

1. **Java JDK 17** (or above)
2. **Eclipse IDE**
3. **MySQL Server + MySQL Workbench**
4. (Optional) Git

Step 1: Get the Project Code

Option A: **Download ZIP** from GitHub and Extract

Option B: **Clone**

git clone <github-repo-link>

Step 2: Import Project in Eclipse

1. Open Eclipse
2. Go to: **File → Import**
3. Select: **Maven → Existing Maven Projects**
4. Click **Next**
5. Browse and select extracted project folder (where pom.xml exists)
6. Click **Finish**
7. Wait for Maven dependencies download

If dependencies not downloaded:

- Right click project → **Maven → Update Project**
- Tick **Force Update**
- Click **OK**

Step 3: Create Database in MySQL

1. Open **MySQL Workbench**
2. Open query tab and run:

CREATE DATABASE role_auth;

Step 4: Configure Database in application.properties

Open file:

src/main/resources/application.properties

Update these values:

spring.datasource.url=jdbc:mysql://localhost:3306/role_auth

spring.datasource.username=root

spring.datasource.password=YOUR_MYSQL_PASSWORD

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

Step 5: Run the Project

In Eclipse:

1. Open main file:
   RoleBasedAuthenticationApplication.java
2. Right click → **Run As → Java Application**

Step 6: Check Application Started

In Eclipse Console you must see:

- Tomcat started on port(s): 8080
- Started RoleBasedAuthenticationApplication

This confirms project is running successfully.

Step 7: Open Application in Browser

Open Chrome and run:

Login Page:

http://localhost:8080/login

➢ Common Issues & Fix that I face during Execution (Important)

1) Port 8080 already in use

Fix:

Stop previous server: Eclipse console → click **Terminate**

OR

Change port in properties:

server.port=8081

Then open: http://localhost:8081/login

2) MySQL Access denied

Fix:

- Correct username/password in application.properties

3) Maven errors

Fix:

- Right click project → Maven → Update Project → Force Update